

Train and debug.

Learn from the curves.

Choose a training setup, diagnose failed learning, and keep a model that earns its validation result.

🕒 20-25 min core reading · 25-35 min practice · 12 short sections

A network can have correct equations and still learn badly. Inputs may have awkward scales, updates may be too large, or a model may fit the training movies while getting worse elsewhere. This chapter turns the arithmetic from Chapter 20 into a training process you can inspect and debug.

WHAT YOU WILL UNDERSTAND

The main question: how do you tell whether a network is learning usefully, find the cause when it is not, and keep the right model?

The practical path: protect the data split, initialize sensibly, run correct updates, read comparable loss curves, and test one change at a time.

Reading path: twelve short sections, three diagrams, a real training workbench, and a Python workbook. Optimizer formulas and framework details are optional.

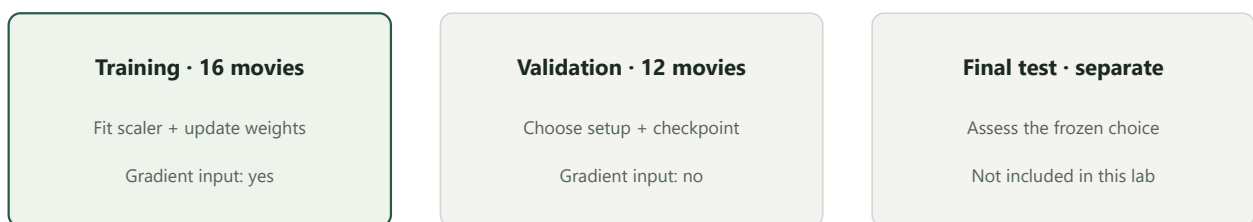
1. Define the experiment before training

Keep the movie preference from Chapter 20: our fictional viewer usually likes a movie dominated by either adventure or romance, but not both. We now use **genre scores** instead of binary flags. Adventure is recorded on a 0-100 scale and romance on a 0-1 scale. They are authored input features, not measured probabilities.

The workbook has **16 training movies and 12 validation movies**, identified as T01-T16 and V01-V12. A high genre score means above the midpoint of its scale. Labels follow the exclusive-OR preference, except training rows T06 and T11, which deliberately disagree. These two exceptions illustrate inconsistent preferences or label noise; they are not secretly corrected during training.

Training data fits preprocessing and supplies gradients. Validation data helps choose hyperparameters and checkpoints. A separate **test set** should assess the final frozen choice. There is no test set in this chapter, so a validation score is never called test performance.

➤ Three data roles; three different permissions



Apply the training-fitted scaler to every split and to inference.

Repeated tuning on validation is expected; it does not create independent test evidence.

Figure 1. Training data fits preprocessing and parameters. Validation data helps choose settings and checkpoints. A separate test evaluates the frozen choice. This chapter includes only training and validation examples.

For real viewing histories, choose splits that match deployment. Predicting a new viewer requires separation by viewer; predicting future behavior may require a time split. Duplicated films, shared users, or future feedback crossing the boundary can make evaluation misleading. The tiny authored splits here teach mechanics, not realistic recommendation quality.

2. Make inputs and initialization suitable for learning

An adventure score of 90 and a romance score of 0.9 describe similarly strong features, but their numerical scales differ by 100. Large sums can push tanh toward -1 or 1 , where its slope is small. **Standardization** subtracts a training mean and divides by a training standard deviation:

$$x'_i = \frac{x_i - \mu_i}{s_i}.$$

Fit μ_i and s_i on training rows only, and apply those same values to validation and new movies. Our training means are (50, 0.5) and standard deviations are approximately (30.2076, 0.3021). Thus adventure 90 becomes about 1.3242. Constant features need a defined policy; the workbook uses a divisor of 1 when the deviation is zero.

Initialization chooses the starting weights. Different hidden units need different starting connections to escape symmetry. With every weight and bias zero in our tanh network, all hidden activations are zero. Output-weight gradients are then zero, and hidden gradients also vanish because the output weights are zero. Only the output bias learns the training like rate.

Our normal preset uses seeded **Xavier initialization**, which adjusts weight scale to the layer's input and output sizes. Zero biases are fine here; making every hidden weight identical is the problematic case. See the [D2L explanation of initialization and numerical stability](#).

▼ Go deeper: Xavier, He, and what a seed does

Our Xavier weights are uniform in $[-a, a]$, where $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$. For the $2 \rightarrow 8$ hidden layer, $a = \sqrt{0.6} \approx 0.7746$. Biases start at zero. **He initialization**, often paired with rectified linear unit (ReLU) activations, commonly uses weight variance $2/n_{\text{in}}$. These scale choices help control signal magnitudes; they do not guarantee good gradients through every architecture.

A random seed makes a particular random sequence reproducible under a specified implementation. Our simple generator uses seed 42 for weights and seed 1000 + epoch for each shuffle. Identical seeds do not promise identical results across arbitrary hardware, libraries, or parallel kernels. Record the implementation and environment as well as the seed.

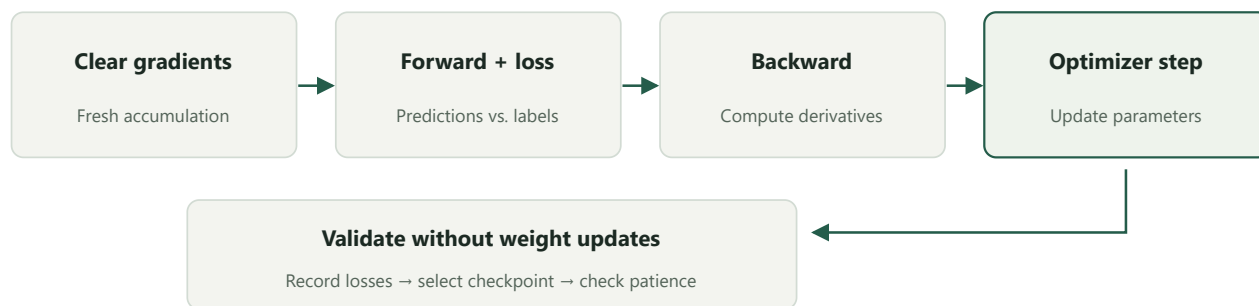
The unscaled, batch-size-4 experiment demonstrates this limitation: small floating-point differences between Python and JavaScript grow into different later trajectories. Each run is reproducible within the checked runtime, but its later numerical values should not be expected to match across them. The full-batch examples quoted in the chapter agree within numerical tolerance.

3. Build a correct training and validation loop

For each training batch, clear old gradient accumulations, compute the forward pass and loss, backpropagate, then update parameters. The manual workbook creates a fresh gradient array each time. Frameworks often accumulate into existing gradient buffers unless you clear them.

Check **shapes and target meaning** before training for hours. A binary classifier can output one logit per movie with one 0/1 target. A loss that already combines sigmoid with binary cross-entropy expects logits; applying another sigmoid first changes the calculation. Silent broadcasting can also compare the wrong pairs of predictions and targets.

➤ Keep the update path separate from validation



Framework evaluation mode and disabling gradient recording are separate controls.

Figure 2. A training batch produces gradients and an optimizer update. Validation uses current weights to measure performance without changing them. Its result can select a checkpoint or trigger a stopping rule.

An **epoch** is one pass over training rows; a **step** is one optimizer update. With 16 rows, batch size 16 gives one step per epoch; batch size 4 gives four. Our shuffle visits each row exactly once per epoch. Mini-batches change gradient noise and update count, so “same epochs” does not mean “same number of updates.” Batch size also affects memory and hardware utilization.

After each epoch, evaluate training and validation data using the **same fixed parameters and evaluation conditions**. Average loss by the number of examples, not by an unweighted mean of differently sized batch means. Our chart shows data loss on both splits; the training regularization penalty is reported separately.

4. Choose an optimizer and a learning-rate policy

An **optimizer** translates gradients into parameter updates. Plain stochastic gradient descent (**SGD**) uses $\theta \leftarrow \theta - \eta g$. With the entire training set in each batch, this becomes full-batch gradient descent. **Momentum** retains a running direction, helping reduce back-and-forth motion. **Adam** uses running averages of gradients and their squares to adapt the effective step for each parameter.

The learning rate η still matters with Adam. Too small can mean little progress within the budget; too large can overshoot, oscillate, or diverge. In our large-rate preset, training loss jumps from **0.7183 to 1.3859 on the first update**, before recovering. A temporary spike is a clue to inspect, not proof that recovery is impossible.

A **schedule** changes the learning rate over training: for example, a short warmup followed by decay, or a reduction when validation stalls. Warmup starts with smaller updates; decay gradually reduces their scale. Record whether a schedule advances per optimizer step or per epoch. Our workbench uses a constant rate so that its comparisons remain easy to trace.

▼ Go deeper: Adam arithmetic and AdamW versus L2

For gradient g_t , Adam maintains $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ and $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$. Starting these averages at zero creates an initial bias, corrected by $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$. The update is

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}.$$

Squares, roots, and divisions act per parameter. The workbook uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Its SGD preset changes only the optimizer, keeping rate 0.03; that controlled comparison does not find the best learning rate for each optimizer.

With **L2 regularization**, a weight penalty contributes λw to the gradient before Adam adapts it. **AdamW** instead applies weight decay separately from that adaptive gradient step. These are generally different updates; the [AdamW paper](#) explains why they should not be treated as interchangeable. Our regularized preset implements Adam with L2, not AdamW. Momentum, schedules, and AdamW are explained here but are not additional workbook modes.

5. Read loss curves as evidence

Read training and validation curves together. **Underfitting** means the learned model has not captured enough useful structure; limited capacity is one cause, but poor optimization, missing features, or an insufficient budget can look similar. **Overfitting** means improvements on training data fail to carry over to the target distribution.

Observed behavior	What to investigate first
Both losses barely improve	Data and labels, broken gradients, initialization, rate, capacity, training budget
Training improves while validation worsens	Evaluation consistency, split differences, noise, excess fitting, checkpoint choice
Loss spikes or becomes non-finite	Input scale, unstable arithmetic, gradients, learning rate
Implausibly excellent validation	Duplicates, leakage, accidental use of validation labels, incorrect metrics

➤ A curve is a clue, not a diagnosis

Both losses stay high

Check data, gradients,
capacity, rate, and budget

Train ↓; validation ↑

Verify the gap, then test
stopping or regularization

Spikes or non-finite loss

Inspect scale, arithmetic,
gradients, and step size

Observe → form one hypothesis → change one thing → compare

A better training score alone is not enough to accept a change.

Figure 3. Verify the measurement before interpreting the pattern. Change one suspected cause, rerun on the same development split, and compare both quality and training cost.

A gap alone does not establish its cause. Dropout, augmentation, or a penalty included only in the training metric can make raw training and validation losses incomparable. Distribution differences can also create a gap. First measure both splits consistently, then test a hypothesis.

Track the task metric as well as the optimized loss. In the large-rate run, validation accuracy is 100% at the selected checkpoint and 91.67% at the final epoch. A confidently wrong prediction can increase log loss substantially while accuracy changes by only one example.

6. Control fitting with suitable regularization

Regularization biases training toward solutions expected to generalize better. Our L2 objective is

$$J = L_{\text{training}} + \frac{\lambda}{2} \sum_{i \in \text{weights}} \theta_i^2.$$

Only weights are penalized here; biases are excluded. The derivative adds $\lambda\theta_i$ for a penalized weight. If $L = 0.4$, the sum of squared weights is 10, and $\lambda = 0.02$, the penalty is 0.1 and the objective is 0.5. The chart still compares the unpenalized data losses.

Other options include a smaller model, more representative data, stopping earlier, **dropout**, and valid **data augmentation**. Dropout randomly masks activations during training. Augmentation changes inputs in ways that preserve the relevant label, such as an appropriate image crop; arbitrary genre-score changes need not preserve a movie preference.

Regularization is not a guaranteed improvement. In this dataset, the chosen L2 setting raises the final validation loss from roughly **0.0477 to 0.2138**. That result argues against accepting this particular setting on these development cases; it does not show that regularization is generally harmful. Do not delete inconvenient labels without checking what they mean.

▼ Go deeper: dropout and normalization are different mechanisms

With inverted dropout rate d , a kept activation is divided by $1 - d$ during training. For $h = 2$ and $d = 0.25$, the output is 0 with probability 0.25 or $2/0.75 \approx 2.6667$ with probability 0.75, preserving expected activation 2. During ordinary evaluation, masking is disabled and the unmodified activation is used. This does not imply that the entire nonlinear network's expected output is unchanged. The [D2L dropout chapter](#) develops the idea.

Batch normalization typically normalizes activations using batch statistics during training, with learned scale and offset. Common implementations maintain running statistics for evaluation. **Layer normalization** normalizes a specified group of features within each example and does not rely on running batch statistics in the same way. Neither replaces the input-data contract or guarantees generalization. Normalization, dropout, and data augmentation are not implemented in this tiny workbench; they are options to investigate when appropriate to a larger model.

7. Debug gradients and numerical problems systematically

Start with one batch you can inspect. Verify feature ranges, targets, output shapes, loss values, and that parameters actually change. Temporarily turn off regularization and try to **overfit a tiny fixed subset**. Failure on a representable simple subset suggests a wiring or optimization problem; success is a useful check, not evidence of generalization.

Look at gradients **per layer**, not just one global number. **Vanishing gradients** become too small to drive useful changes; repeated small derivatives and saturated activations can contribute. **Exploding gradients** become very large and destabilize updates. Our all-zero preset has a nonzero output-bias gradient while its hidden weights stay frozen, so a nonzero total norm is insufficient proof that every layer learns.

The workbench reports the Euclidean norm of the full-training mean objective gradient and the proportion of hidden activations with absolute value above 0.99. These diagnostics describe magnitude and saturation; they are not universal pass/fail thresholds.

Gradient clipping caps a gradient's norm before an update, for example $g' = g \min(1, c/\|g\|)$ for a positive cap c and nonzero g . A gradient $(3, 4)$ has norm 5; clipping to 2 gives $(1.2, 1.6)$. Clipping can limit damaging updates, but cannot repair wrong labels, leakage, or a broken loss.

▼ Go deeper: non-finite values and mixed precision

NaN means "not a number"; infinities can arise from overflow or division by zero. Find the first non-finite input, activation, loss, gradient, or parameter rather than replacing the final NaN with zero. Stable logit-based cross-entropy avoids some avoidable overflows, but cannot cure every source of invalid arithmetic.

Mixed precision uses lower-precision arithmetic for selected operations to reduce memory and potentially improve throughput, while preserving higher precision where needed. Some lower-precision formats benefit from loss scaling to reduce gradient underflow. If gradients are scaled, unscale them before norm clipping; otherwise the cap is applied to the wrong magnitude. Hardware, numerical format, and framework support determine the details. This workbook uses ordinary floating-point arithmetic and does not implement clipping or mixed precision.

8. Select checkpoints and stop with a clear rule

A **checkpoint** saves model state at a particular time. The last model is not necessarily the most useful one. Our run records validation loss after every epoch, including initialization, and keeps the parameters with the **lowest observed validation loss**. Exact ties keep the earlier checkpoint.

Early stopping ends training after a chosen period without meaningful validation improvement. Here patience is 30 epochs, and its improvement threshold is 0.0001 in absolute loss. Patience resets when loss beats the previous patience reference by more than that amount. Checkpoint selection is separate: it still tracks every strict improvement, even a smaller one.

For the large-rate full-batch run, 600 epochs reveal a minimum at **epoch 112**. Enabling patience 30 stops at **epoch 46** and selects **epoch 16**. The rule cannot know that another recovery would occur later. Choose the policy on development data, and distinguish the stopped epoch from the selected epoch. The workbook returns both snapshots; select the best snapshot explicitly for inference.

Save preprocessing, feature order, architecture, weights, configuration, and data/version identifiers. To resume training, also preserve optimizer state, counters, scheduler state if used, and random/shuffle state or its reproducible rule. Saving only weights can support inference but does not fully describe a training continuation.

▼ Go deeper: the PyTorch train/eval boundary

In PyTorch, `model.train()` and `model.eval()` control modules with different training/evaluation behavior, such as dropout and common batch-normalization configurations. `torch.no_grad()` separately disables gradient recording. Evaluation mode alone does not do that. See the official [autograd mechanics note](#) and [optimization tutorial](#).

This illustrative pattern assumes a binary model that returns logits, constructed data loaders, a configured optimizer, and inputs on the correct device. It is not the standard-library workbook and is not executed by the website:

```

model.train()
for x, y in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(x).squeeze(-1)
    loss = F.binary_cross_entropy_with_logits(logits, y.float())
    loss.backward()
    optimizer.step()

model.eval()
total, count = 0.0, 0
with torch.no_grad():
    for x, y in validation_loader:
        logits = model(x).squeeze(-1)
        total += F.binary_cross_entropy_with_logits(
            logits, y.float(), reduction="sum"
        ).item()
        count += y.numel()
validation_loss = total / count # require a nonempty loader

```

Here `F` denotes `torch.nn.functional`. The loss uses a sum during validation so dividing by the total example count gives the correct mean even if the last batch is smaller. Intentional gradient accumulation across training batches requires explicit loss scaling and a defined update schedule; forgetting to clear gradients is not that design.

9. Investigate the training workbench

Start with the large-rate experiment. Switch between the best validation checkpoint and the last epoch, then enable early stopping. Compare the selected epoch with the stopping epoch. Next try unscaled inputs and inspect the **initial** saturation, then compare the scaled baseline. Finally try all-zero initialization and look beyond the total gradient norm.

Lower training loss. Worse validation.

A real $2 \rightarrow 8 \rightarrow 1$ tanh network learns fictional genre preferences. Six presets change one baseline setting. The curves are computed locally; no pretrained model or service is involved.

Batch size 16; 600-epoch budget; early stopping off. Each row changes one baseline setting. “Best” means the lowest validation loss among the epochs observed.

Preset	Best epoch	Best validation loss	Final validation loss
Scaled + Adam	600	0.0477	0.0477
Unscaled inputs	169	0.4186	0.4910
All-zero initialization	0	0.6931	0.7254
Large Adam learning rate	112	0.0419	0.1899
L2-regularized Adam	600	0.2138	0.2138
Plain gradient descent	600	0.4272	0.4272

The large-rate run selects epoch 112 (validation loss 0.0419), while epoch 600 has loss 0.1899 and validation accuracy 91.67%. Training loss still falls. With patience 30 enabled, that run stops at epoch 46 and selects epoch 16 (loss 0.0738); a stopping rule cannot see the later recovery.

At initialization, unscaled inputs saturate 85.16% of hidden activations versus 0% with scaling. The all-zero model learns only an output bias. This particular L2 setting worsens validation loss compared with the baseline. These are authored development cases, not a ranking of optimizers or production-quality estimates.

The network has two inputs, eight tanh units, and one sigmoid output: **33 parameters**. Initialization, epoch shuffles, and optimizer settings are fixed within each preset. Changing a control starts a fresh run. The browser performs actual training and evaluates both splits after every epoch.

Changing batch size to 4 increases updates per epoch from one to four. The workbench displays both counts; it does not report measured time or claim that one setting is universally faster or better. Validation is used repeatedly for these comparisons, so the results remain development evidence.

10. Reproduce the run and practice diagnosing it

The four-file workbook contains `data.json`, `train.py`, `test_train.py`, and `README.md`. Python 3.10+ and its standard library are sufficient. It implements scaling, Xavier initialization, manual backpropagation, SGD, Adam, L2, deterministic mini-batches, checkpoint selection, and optional early stopping.

```
py train.py --preset fast
py train.py --preset fast --early-stop
py train.py --preset raw --batch-size 4
py train.py --preset baseline --epochs 100
py -m unittest -v test_train.py
```

Use `python` or `python3` if needed. Commands print JSON, make no network calls, and save no model automatically. `--input` accepts a compatible dataset. Invalid arguments return code 2. Read `best` and `last` as separate parameter snapshots; `history` records comparable metrics at every epoch. `README.md` shows how to predict using the chosen snapshot and saved scaler.

Exercise 1: count the actual updates

You train on 16 movies for 50 epochs. How many updates occur with batch sizes 16 and 4? Does a fourfold update count prove a fourfold runtime?

▼ Reveal the reasoning

Batch 16 gives 50 updates; batch 4 gives 200. Both process each training row 50 times, but smaller batches update more often. Runtime also depends on batch overhead and hardware utilization, so update count alone does not prove a runtime ratio.

Exercise 2: separate regularization from data loss

Mean training data loss is 0.4, the sum of squared penalized weights is 10, and $\lambda = 0.02$. What objective is optimized? What does L2 add to the gradient of a weight equal to 3? Should that penalty be added to the validation curve here?

▼ Reveal the reasoning

The penalty is $(0.02/2) \times 10 = 0.1$, so the objective is 0.5. The extra gradient is $0.02 \times 3 = 0.06$. Our validation curve measures only data loss, and the training curve uses that same definition; the penalty is reported separately.

Exercise 3: restore the selected checkpoint

A run stops at epoch 46, but its lowest observed validation loss occurred at epoch 16. Which weights should the selected predictor use? What does a later minimum at 112 in a separate longer run tell you about patience?

▼ Reveal the reasoning

Use the saved epoch-16 weights with the saved training-fitted scaler. Epoch 46 is the stopping point, not the selected predictor. The longer run shows that patience can stop before a later improvement. It cannot predict the future or guarantee the best possible checkpoint.

11. Check your understanding

01 Which rows may determine feature scaling and weight gradients?

- A. Training rows only.
- B. Training and validation rows together.
- C. Whichever rows give the lowest loss.

Answer: A. Fit preprocessing on training data and reuse it on validation. Validation can select a setup or checkpoint, but it must not supply training gradients.

02 Training loss falls while comparable validation loss rises. What is the next useful step?

- A. Assume every weight gradient is zero.
- B. Inspect overfitting, split differences, and evaluation correctness before choosing a remedy.
- C. Use the test labels for another update.

Answer: B. The gap is a diagnostic clue. Check the measurement and data boundary, then test a focused change such as early stopping or suitable regularization.

03 What does `model.eval()` do in PyTorch?

- A. Automatically disables gradient tracking.
- B. Permanently deletes optimizer state.
- C. Changes behavior of modules such as dropout and batch normalization; `no_grad()` is separate.

Answer: C. Evaluation mode and gradient tracking are independent. Validation usually needs both evaluation behavior and no gradient recording.

04 With 16 rows and batch size 4, how many optimizer updates occur in one epoch here?

- A. 1.
- B. 4.
- C. 16, regardless of batch size.

Answer: B. Each batch of four produces one update, so $16 / 4 = 4$. Comparisons at equal epochs then use different update counts than full-batch training.

05 Early stopping selected epoch 16 and stopped at 46. A longer run later improved at 112. What does this show?

- A. The checkpoint code must always be wrong.
- B. Patience guarantees the globally best model.
- C. The rule chooses among observed epochs and can stop before a later recovery.

Answer: C. A stopping policy cannot observe future improvements. Tune it on development data, preserve the selected checkpoint, and confirm the final policy separately.

12. Make every change an experiment

Write down the observation, proposed cause, single change, and acceptance criterion before comparing runs. Preserve the data split and record the seed, architecture, optimizer, schedule, batch size, epoch/update counts, checkpoint rule, and both data losses. Include task metrics and relevant compute cost; a small loss change alone may not justify a much larger budget.

Use several seeds and suitable uncertainty analysis when judging meaningful improvements on real data. Repeatedly inspecting one validation split can overfit your selection process. Once the training policy is settled, freeze it and use independent confirmation rather than calling the best development result a final test score.

Chapter 22: CNNs, sequence models, and transfer learning will explore architectures suited to images and sequences, and how to reuse a pretrained model rather than learning every representation from scratch.

▼ Quick-reference glossary

Term	Meaning here
Initialization / seed	Starting parameter values / reproducible random-sequence setting
Batch / step / epoch	Examples in an update / one update / one pass over training rows
Optimizer / schedule	Rule for parameter updates / rule for changing the learning rate
Momentum / Adam	Retained gradient direction / adaptive updates using gradient moments
Data loss / objective	Prediction error / prediction error plus a training penalty
Regularization / dropout	Constraints favoring useful fitting / random activation masking in training
Saturation / gradient norm	Activation near a flat extreme / magnitude of a gradient vector
Clipping / mixed precision	Limit gradient magnitude / use selected numerical precisions
Checkpoint / patience	Saved model state / tolerated span without a specified improvement
Evaluation mode / no-grad	Module behavior switch / gradient-recording switch

Sources and further reading

The D2L initialization and dropout chapters, AdamW paper, and official PyTorch documentation are linked beside the related ideas. The movie data, presets, diagrams, workbook, and measured traces are original teaching material. The workbench uses no PyTorch installation, external model, or independent test set.